


# University of Asia Pacific

Department of Computer Science and Engineering

## CSE 402: Operating System Lab

### Lab Report: 07

## Comparative Study of FIFO and MRU Page Replacement Algorithms

**Submitted by:**

**Name:** Faiaz Masnun Labib
**Student ID:** 231011218
**Section:** C-1

**Submitted to:**

**Atia Rahman Orthi**
Lecturer,
Department of CSE, UAP

**Date of Submission:** October 19, 2026

# 1. Problem Specification

- **Objective:** Implement, trace, and contrast the First-In, First-Out (FIFO) and Most Recently Used (MRU) page replacement algorithms under a fixed frame capacity.
- **Input Parameters:**
  - Reference String ($P$): $[1, 2, 3, 4, 5, 1, 2, 3, 1]$
  - Total Page Requests ($N$): 9
  - Frame Capacity ($M$): 3 Frames
- **Evaluation Metrics:**
  - Page Hit (Hit): Requested page is already resident in main memory.
  - Page Fault / Miss (Fault): Requested page is absent from memory, requiring a page fetch.
  - Hit Ratio (HR) = $\frac{\text{Total Hits}}{N}$
  - Fault Ratio (FR) = $\frac{\text{Total Faults}}{N} = 1 - \text{HR}$

| **Step** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| **Page Reference** | 1 | 2 | 3 | 4 | 5 | 1 | 2 | 3 | 1 |

Table 1: Reference String Sequence

# 2. Source Code

```python
def fifo(pages: list[int], capacity: int):
    memory = []
    hit = 0
    fault = 0
    for p in pages:
        if p in memory:
            hit += 1
            s = "Hit"
        else:
            fault += 1
            s = "Miss"
            if len(memory) == capacity:
                memory.pop(0)
            memory.append(p)
    return hit, fault, hit / len(pages)


def mru(pages: list[int], capacity: int):
    memory = []
    hit = 0
    fault = 0
    for p in pages:
        if p in memory:
            hit += 1
            s = "Hit"
```

```
            memory.remove(p)
            memory.append(p)
        else:
            fault += 1
            s = "Miss"
            if len(memory) == capacity:
                memory.pop()
            memory.append(p)
    return hit, fault, hit / len(pages)


refString = [1, 2, 3, 4, 5, 1, 2, 3, 1]
frameSize = 3

fifo_hit, fifo_fault, fifo_hr = fifo(refString, frameSize)
mru_hit, mru_fault, mru_hr = mru(refString, frameSize)

print("FIFO -> Hits:", fifo_hit, "Faults:", fifo_fault, f"Hit Ratio: {fifo_hr:.4f}")
print("MRU  -> Hits:", mru_hit, "Faults:", mru_fault, f"Hit Ratio: {mru_hr:.4f}")
```

**GitHub Repository Link:** Click Here

# 3. Step-by-Step Execution Traces & Quantitative Results

## A. FIFO Page Replacement Trace

- When a page fault occurs and frames are full, the page loaded earliest (index 0) is victimized.

| Step | Page Ref | Memory Frames (Oldest → Newest) | Status | Replaced Page |
|---|---|---|---|---|
| 1 | 1 | [1] | Miss | None (Compulsory) |
| 2 | 2 | [1, 2] | Miss | None (Compulsory) |
| 3 | 3 | [1, 2, 3] | Miss | None (Compulsory) |
| 4 | 4 | [2, 3, 4] | Miss | 1 |
| 5 | 5 | [3, 4, 5] | Miss | 2 |
| 6 | 1 | [4, 5, 1] | Miss | 3 |
| 7 | 2 | [5, 1, 2] | Miss | 4 |
| 8 | 3 | [1, 2, 3] | Miss | 5 |
| 9 | 1 | [1, 2, 3] | **Hit** | None |
| **Total Hits:** 1 — **Total Faults (Misses):** 8 — **Hit Ratio:** 11.11% (0.1111) | | | | |

## B. MRU Page Replacement Trace

- When a page fault occurs and frames are full, the most recently accessed/inserted page is victimized (memory.pop()).

| Step | Page Ref | Memory Frames (LRU → MRU) | Status | Replaced Page |
|---|---|---|---|---|
| 1 | 1 | [1] | Miss | None (Compulsory) |
| 2 | 2 | [1, 2] | Miss | None (Compulsory) |
| 3 | 3 | [1, 2, 3] | Miss | None (Compulsory) |
| 4 | 4 | [1, 2, 4] | Miss | 3 |
| 5 | 5 | [1, 2, 5] | Miss | 4 |
| 6 | 1 | [2, 5, 1] | **Hit** | None (Moved to MRU) |
| 7 | 2 | [5, 1, 2] | **Hit** | None (Moved to MRU) |
| 8 | 3 | [5, 1, 3] | Miss | 2 |
| 9 | 1 | [5, 3, 1] | **Hit** | None (Moved to MRU) |
| **Total Hits:** 3 — **Total Faults (Misses):** 6 — **Hit Ratio:** 33.33% (0.3333) | | | | |

## C. Comparative Benchmark Summary

| Algorithm | Total Hits | Total Page Faults | Hit Ratio | Fault Ratio |
|---|---|---|---|---|
| FIFO | 1 | 8 | 11.11% (0.1111) | 88.89% (0.8889) |
| MRU | 3 | 6 | **33.33% (0.3333)** | **66.67% (0.6667)** |

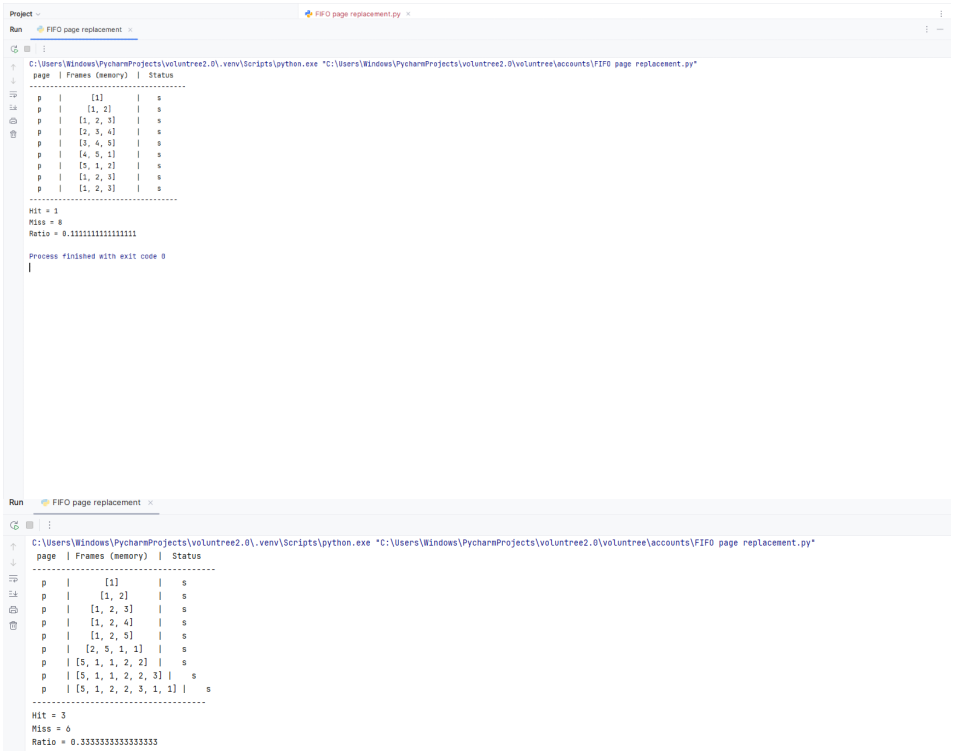



Figure 1: Program Execution Output Verification

# 4. Comparative Evaluation: Which Approach Performs Better?

**Decision: MRU (Most Recently Used) is significantly better** than FIFO for this reference sequence.

- **Quantitative Difference:**

$$\Delta\text{Hits} = 3 - 1 = +\mathbf{2} \text{ Hits (300\% of FIFO hits)}$$
$$\Delta\text{Faults} = 8 - 6 = \mathbf{2} \text{ Faults reduced (25\% reduction)}$$
$$\Delta\text{Hit Ratio} = 33.33\% - 11.11\% = +\mathbf{22.22}\%$$

- **Point-to-Point Analysis:**

1. **FIFO Blind Eviction:** FIFO replaces pages based strictly on arrival order, without regard to frequency or future reuse. Pages 1 and 2, which arrived first, are repeatedly evicted right before they are needed again in steps 6 and 7.
2. **MRU Working Set Preservation:** By repeatedly evicting the newest incoming pages (steps 4 and 5 replace 3 and 4), MRU preserves the older pages (1 and 2) in the frame base. When pages 1 and 2 are subsequently referenced in steps 6, 7, and 9, they result in hits.

# 5. Discussion

- **Locality vs. Looping Workloads:** While Least Recently Used (LRU) generally outperforms MRU under temporal locality, MRU excels in cyclic access patterns where the reference string loops over a dataset larger than the frame pool. In such scenarios, MRU preserves older reference anchors while discarding transients.
- **FIFO Belady's Anomaly:** FIFO treats main memory as a strict queue. It is vulnerable to Belady's Anomaly (where increasing frame capacity can paradoxically increase page faults) and exhibits poor locality utilization.
- **Conclusion:** For this specific reference string, MRU yields three times the hit count of FIFO (3 hits vs. 1 hit) and reduces page faults by 25%. Hence, MRU provides superior memory management for this access pattern.